UNIVERSITY OF TEHRAN

College of Engineering

School of Electrical and Computer Engineering

DESIGN AND IMPLEMENTATION OF AN ESP32-BASED NTC THERMOMETER

NTC Linearization, Analog Signal Conditioning,
OLED Display, and Bluetooth Low Energy Monitoring

TECHNICAL PROJECT REPORT

Project Team

Mohammad Mehdi Fazlollahpour | 810100195

Course: Electronic Systems Implementation
Instructor: Dr. Ashtiani
June 2024

Report Contents

Section	Coverage
Executive Summary	Project purpose and principal outcome
1. Introduction	Background, motivation, and scope
2. System Overview	Functional architecture and operating sequence
3. Hardware Design	NTC model, linearization, signal conditioning, PCB, and components
4. Embedded Firmware	ADC conversion, OLED interface, BLE service, and threshold logic
5. Mobile Application	MIT App Inventor interface and BLE interaction
6. Design Evidence and Results	Simulation plots, schematic, PCB, and application evidence
7. Limitations and Future Improvements	Verification gaps and recommended engineering work
8. Conclusion	Summary of the implemented system
Appendix A	Firmware configuration and key code excerpts
Appendix B	Technical verification notes for the project team
Source Materials	Project documentation and implementation artifacts

Scope and evidence statement

This report preserves the design values and implementation approach documented in the supplied Persian report and ESP32 source code. The source package contains circuit simulations, schematic and PCB views, and mobile-app screenshots. It does not provide a calibrated measurement dataset or quantitative accuracy results; therefore, no measurement-accuracy claim is made here.

Executive Summary

This project presents the design and implementation of an electronic thermometer based on an NTC thermistor and an ESP32 microcontroller. Because the resistance–temperature characteristic of an NTC thermistor is nonlinear, the sensing element is linearized over an intended operating interval of approximately 1–100 °C by placing a fixed resistor in parallel with the thermistor. The resulting resistance is converted into a voltage through a resistive divider and buffered before being sampled by the ESP32 analog-to-digital converter (ADC).

The firmware converts the ADC reading into temperature using the linear transfer equation included in the supplied code. The measured temperature is presented on a 128 × 64 OLED display and transmitted to a mobile device through Bluetooth Low Energy (BLE). A writable BLE characteristic receives a user-defined temperature threshold. Two LEDs provide a local status indication: the red LED is activated when the measured temperature is less than or equal to the threshold, while the green LED is activated when the temperature is above it. The project also includes an Altium Designer schematic and PCB layout, as well as a mobile interface developed with MIT App Inventor.

The report documents the sensing principle, analog front-end calculations, embedded-software structure, BLE data exchange, and available design evidence. It also identifies implementation details that should be verified before the project is presented as a fully validated measurement instrument.

Keywords: NTC thermistor; temperature measurement; thermistor linearization; ESP32; ADC; OLED; Bluetooth Low Energy; Altium Designer; MIT App Inventor.

1. Introduction

Temperature measurement is a common requirement in electronic instrumentation, monitoring, and control systems. NTC thermistors are attractive sensing elements because their resistance changes significantly with temperature and they are available in compact, low-cost packages. Their principal signal-conditioning challenge is the nonlinear relationship between resistance and temperature. A measurement system that directly samples the thermistor voltage therefore requires an appropriate conversion model or an analog linearization method.

The project addresses this challenge through a combined hardware–software approach. A parallel resistor is selected to reduce the curvature of the thermistor response around the midpoint of the intended temperature range. A voltage-divider stage converts the conditioned resistance into an analog voltage, while a buffer is included to reduce loading by the following measurement stage. An ESP32 then performs the temperature conversion and provides local and wireless outputs.

1.1 Project Objectives

- Develop an NTC-based temperature-sensing front end with an improved, approximately linear response over the intended temperature interval.
- Scale the sensor signal to a voltage compatible with the ESP32 ADC input range.
- Design the circuit schematic and printed circuit board (PCB) using Altium Designer.
- Implement continuous temperature acquisition and conversion on the ESP32.
- Display the measured temperature on a 128 × 64 OLED module.
- Transmit temperature readings to a mobile application using BLE notifications.
- Allow the mobile application to configure a temperature threshold and indicate the threshold state using two LEDs.

1.2 Project Scope

The documented scope includes the NTC linearization calculations, analog signal-conditioning concept, circuit simulation, schematic and PCB design, ESP32 firmware, BLE communication, OLED display, and a mobile interface. The supplied material does not include a formal calibration procedure, a reference-instrument comparison, uncertainty analysis, or a complete environmental test campaign.

2. System Overview

The thermometer is organized as a sequence of sensing, conditioning, conversion, processing, and presentation stages. The NTC thermistor provides a temperature-dependent resistance. The parallel linearization resistor modifies the resistance curve, and the divider converts the resulting resistance into a voltage. A buffer isolates the divider from the subsequent input stage. The ESP32 samples the analog signal and executes the temperature-conversion and output routines.

Subsystem	Function	Documented implementation
Sensing element	Temperature-dependent resistance	10 kΩ NTC thermistor; β = 3950 K
Linearization	Reduce resistance-curve nonlinearity	Parallel resistor; selected value Rp = 3.9 kΩ
Analog conditioning	Convert resistance to a bounded voltage and isolate the node	5 V divider, calculated R1 ≈ 2.77 kΩ, op-amp buffer
Controller	ADC acquisition and system control	ESP32; 12-bit ADC reading
Local display	Show temperature to the user	0.96-inch, 128 × 64 OLED; I^2C address 0x3C
Wireless interface	Send temperature and receive threshold	BLE GATT service with notify and read/write characteristics
Status indication	Show threshold state locally	Red and green LEDs

Mobile interface	Display temperature and configure threshold	MIT App Inventor application

2.1 Operating Sequence

1. The analog output of the NTC conditioning circuit is sampled at the ESP32 input assigned to GPIO 13.
2. The ADC count is scaled using the 3.3 V reference assumption and converted to temperature using the linear equation in the firmware.
3. The temperature is printed to the serial interface and displayed on the OLED.
4. The firmware compares the measured temperature with the threshold value received through the writable BLE characteristic.
5. The red or green LED is activated according to the comparison result.
6. When a BLE central device is connected, the ESP32 sends the temperature through a notification characteristic.
7. The mobile application displays the received temperature and allows the user to update the threshold.

3. Hardware Design

3.1 NTC Thermistor Model

The supplied design uses a 10 kΩ NTC thermistor with a beta constant of 3950 K. The nominal resistance reference used in the calculations is 10 kΩ at 300 K. The resistance at an absolute temperature T is represented by the beta model:

$$R_t = R_0 \cdot \exp[\beta \cdot (1/T - 1/T_0)]$$

where R_t is the thermistor resistance at temperature T, R_0 is the reference resistance, β is the material constant, and T_0 is the reference temperature. Absolute temperature must be expressed in kelvin when using this model.

Parameter	Value used in the project
Reference resistance, R_0	10 kΩ
Reference temperature, T_0	300 K
Beta constant, β	3950 K
Intended temperature interval	Approximately 1–100 °C
Nominal midpoint used for calculation	50 °C (323 K); interval midpoint is 50.5 °C

3.2 Parallel-Resistor Linearization

To reduce the curvature of the NTC resistance characteristic, the thermistor is connected in parallel with a fixed resistor Rp. The equivalent resistance is:

$$R_{total} = R_p \parallel R_t = (R_p \cdot R_t) / (R_p + R_t)$$

The design method selects Rp so that the second derivative of the equivalent resistance with respect to temperature is zero at a chosen midpoint Tm. The expression derived in the supplied report is:

$$R_p = R_{tm} \cdot (\beta - 2T_m) / (\beta + 2T_m)$$

For Tm = 50 °C = 323 K, the thermistor resistance calculated from the beta model is approximately 3.915 kΩ. Substitution into the linearization expression gives an initial theoretical parallel-resistor value of approximately 2.815 kΩ. The project report then describes a simulation-and-adjustment step: Rp was changed to 3.9 kΩ because the resulting plotted points were closer to a straight-line fit over the selected interval.

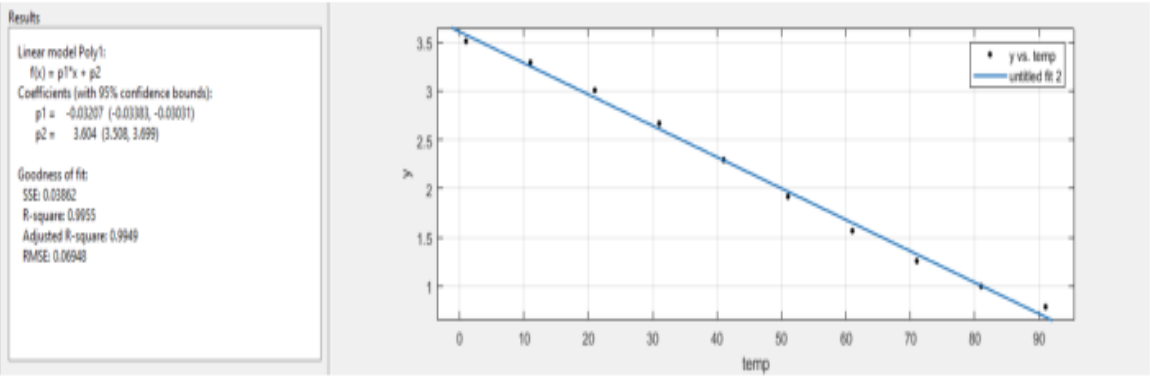


Figure 1. Linearized response after selecting Rp = 3.9 kΩ; reproduced from the supplied project report.

Interpretation

The supplied report demonstrates a qualitative improvement in linearity after the resistor was adjusted to 3.9 kΩ. It does not provide a numerical linearity error, regression residual, or maximum deviation, so the improvement is described qualitatively rather than as a quantified performance result.

3.3 Resistance-to-Voltage Conversion

The linearized resistance is converted into a voltage using a resistive divider. In the project notation, R2 represents the parallel combination of the NTC thermistor and Rp, while R1 is the series resistor. The divider output is:

$$Vout = [R_2 / (R_1 + R_2)] \cdot Vin$$

The design uses a 5 V input supply. Since the ESP32 ADC input range is treated in the project as 0–3.3 V, the divider is designed to keep the output below this limit. The report targets approximately 2.8 V at 0 °C, where the NTC resistance is calculated as 36.77 kΩ.

$$R_2 = 36.77\ k\Omega \parallel 3.9\ k\Omega \approx 3.526\ k\Omega$$

$$2.8\ V = [3.526\ k\Omega / (R_1 + 3.526\ k\Omega)] \cdot 5\ V$$

Solving the divider equation gives R1 ≈ 2.77 kΩ. This is the calculated value documented in the original report. The final fitted resistor value and tolerance should be confirmed against the assembled board before publication.

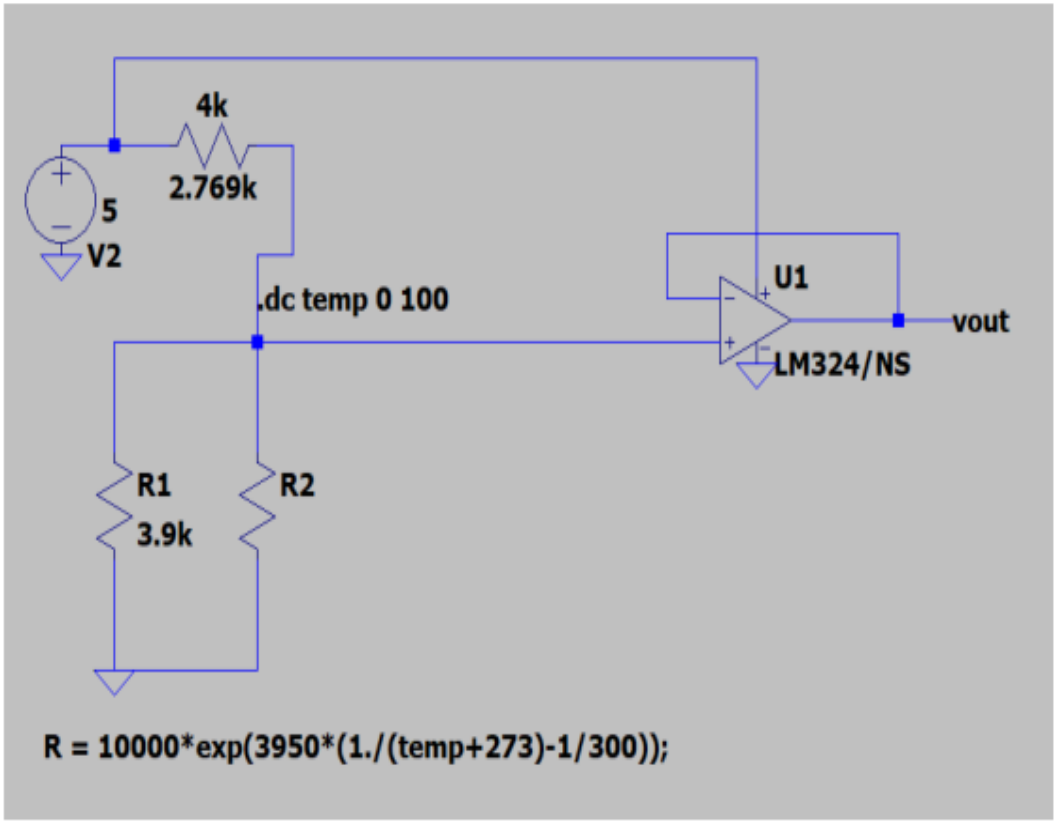


Figure 2. Analog conditioning simulation showing the divider and buffer stage; reproduced from the supplied project report.

3.4 Buffer Stage

A voltage-follower buffer is included after the divider to reduce loading of the sensor-conditioning network by the following measurement input. The original report identifies an LM324 in the simulation diagram and describes the use of a single-supply, 5 V operational amplifier. A separate component-selection section names the LM358. This difference is retained as an item requiring confirmation; the exact operational-amplifier part fitted to the physical board is not unambiguously established by the supplied documentation.

3.5 Schematic and PCB Design

The project report includes an Altium Designer schematic showing the ESP32 module, OLED interface, analog sensing and conditioning connections, indicator LEDs, and external connectors. The PCB illustrations show the routed board, component placement, and a three-dimensional board view. The design also documents a 100 nF, 1206-package capacitor for supply decoupling or filtering, 1% resistors, an XH connector for the NTC, and two input-power options: a pin header and a Micro-USB connector.

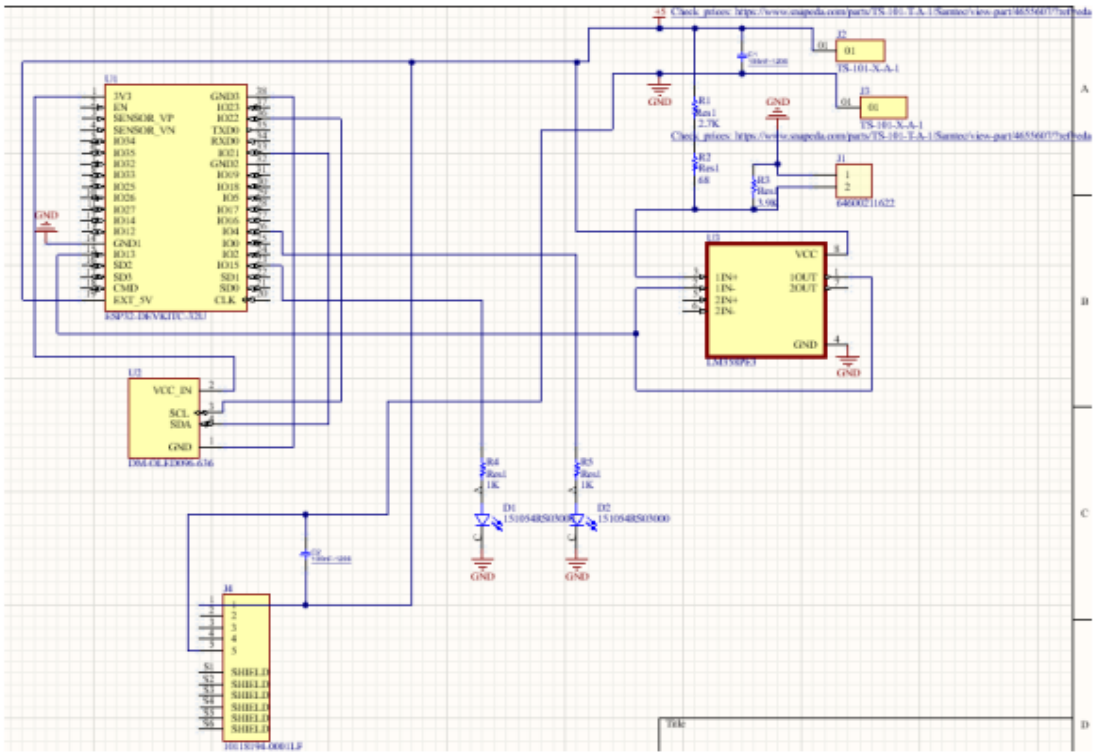


Figure 3. Overall circuit schematic reproduced from the original Altium design documentation.

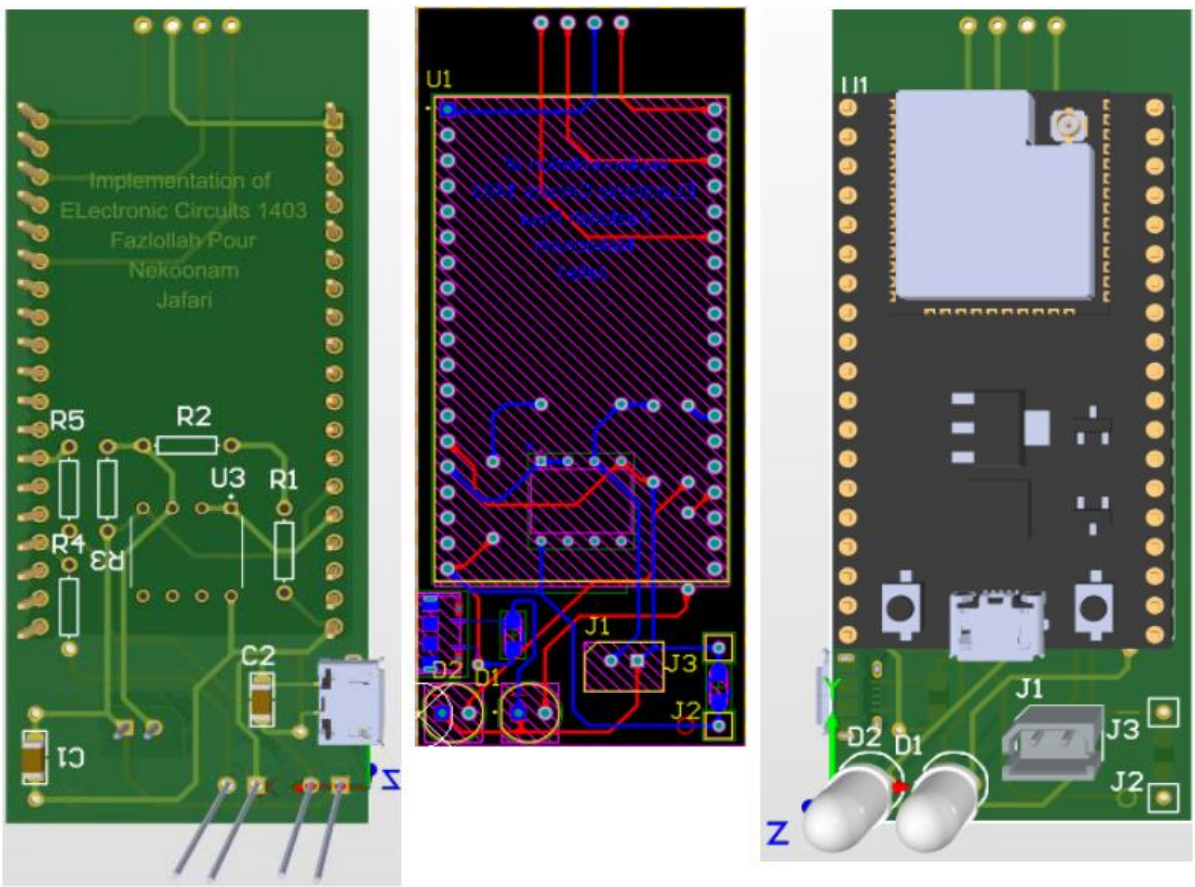


Figure 4. PCB routing and 3D board views reproduced from the supplied project report.

3.6 Main Hardware Components

Component	Role in the design	Details documented
ESP32 module	Main controller and BLE radio	ADC acquisition, processing, display control, and BLE communication
NTC thermistor	Temperature sensing	10 kΩ reference resistance; β = 3950 K
OLED module	Local visual output	DM-OLED096-636; 0.96 inch; 128 × 64 pixels
Operational amplifier	Voltage buffer	LM324 shown in simulation; LM358 named in component discussion
Resistors	Linearization and voltage divider	Rp = 3.9 kΩ; calculated R1 ≈ 2.77 kΩ; 1% resistors are specified
Capacitor	Supply decoupling / filtering	100 nF, 1206 SMD package
Connectors	Sensor and power connections	XH connector, pin header, and Micro-USB input
Indicator LEDs	Threshold status	Red LED on GPIO 15; green LED on GPIO 4

4. Embedded Firmware

The ESP32 firmware is written in Arduino-style C++ and uses the Wire, Adafruit_GFX, Adafruit_SSD1306, and ESP32 BLE libraries. The program initializes the serial interface, GPIO pins, OLED display, BLE server, GATT service, and characteristics. The main loop repeatedly reads the analog input, calculates temperature, updates the indicators and OLED, and sends BLE notifications when a central device is connected.

4.1 Pin Assignment and Peripherals

Signal / peripheral	Firmware configuration	Purpose
GPIO 13	opout; INPUT	Analog sensor-conditioning voltage input
GPIO 15	led1; OUTPUT	Red threshold-status LED
GPIO 4	led2; OUTPUT	Green threshold-status LED
OLED	128 × 64; I^2C address 0x3C	Temperature display
Serial	9600 bit/s	Diagnostic temperature output
BLE device name	ESP32	Advertised peripheral name

4.2 ADC Acquisition and Temperature Conversion

The firmware reads the sensor input using analogRead(opout). The supplied project report describes the ESP32 ADC result as a 12-bit value in the range 0–4095. The code then applies the following linear conversion:

$$T(°C) = -53.561 \cdot (NADC \cdot 3.3 / 4096) + 149.97$$

Here, NADC denotes the raw ADC count returned by analogRead(). Although the source code variable is named voltage, it contains the ADC count before scaling; the multiplication by 3.3/4096 converts that count to the voltage quantity used by the linear equation. The coefficients are reproduced exactly from the supplied firmware. No calibration dataset or derivation of these coefficients is included in the supplied materials.

4.3 OLED Display and Serial Output

The OLED is initialized through the Adafruit_SSD1306 library using the I^2C address 0x3C. The firmware clears the display, writes a temperature label, prints the calculated temperature, and displays the unit text "centigrade". The serial monitor is initialized at 9600 bit/s and receives the calculated temperature. The supplied firmware does not print the raw ADC count or the intermediate sensor voltage.

4.4 BLE GATT Communication

The ESP32 operates as a BLE peripheral and creates a GATT server with one service and two characteristics. The first characteristic is configured for notifications and is used to transmit the measured temperature. The second characteristic supports read and write operations and is used to receive the threshold value from the mobile application.

GATT item	UUID	Properties / purpose
Service	4fafc201-1fb5-459e-8fcc-c5c9c331914b	Groups the thermometer characteristics
Temperature characteristic	beb5483e-36e1-4688-b7f5-ea07361b26a8	Notify; sends the integer temperature value
Threshold characteristic	3da2e68d-283c-4617-b03e-160835de5d5d	Read and write; receives the threshold value

A server callback tracks BLE connection and disconnection events. When the threshold characteristic receives a write, a characteristic callback converts the received value to a string and then to an integer, storing it in the global variable pChar2_value_int. When a BLE central is connected, the firmware assigns the calculated integer temperature to the notification characteristic and calls notify(). After a disconnection, the code restarts advertising so that another central device can connect.

4.5 Threshold Logic and LED Indication

The comparison is implemented as follows: when the measured temperature is less than or equal to pChar2_value_int, the red LED (GPIO 15) is turned on and the green LED (GPIO 4) is turned off. Otherwise, the green LED is turned on and the red LED is turned off. The threshold value is updated through the BLE write characteristic.

```
if (temperature <= pChar2_value_int) {
    digitalWrite(led1, HIGH);  // Red LED
    digitalWrite(led2, LOW);   // Green LED
} else {
    digitalWrite(led2, HIGH);
    digitalWrite(led1, LOW);
}
```

The loop includes delays of 500 ms and 1000 ms around the display-refresh sequence. Consequently, the nominal update cycle is on the order of 1.5 seconds, excluding processing overhead. Temperature acquisition and local indication continue when no BLE central is connected; BLE notifications are issued only while a central is connected.

5. Mobile Application

The mobile interface is documented as being developed with MIT App Inventor and a Bluetooth extension. The application provides a scanning control, connection status, a disconnect control, a temperature field, a threshold field, and a slider for threshold adjustment. The application communicates with the ESP32 through the service and characteristic UUIDs defined in the firmware.

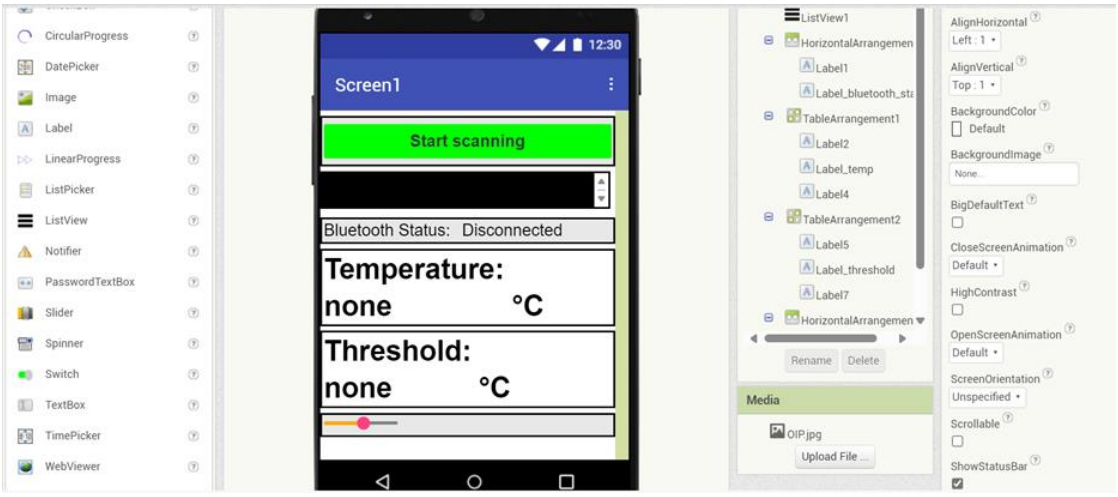


Figure 5. MIT App Inventor interface layout showing scanning, connection status, temperature, and threshold controls.

The supplied report also includes the App Inventor block logic and a screenshot of the running interface. The documented interaction sequence starts scanning when the user selects the scan control, updates the visible controls according to connection state, and allows the slider to send a threshold value to the ESP32. The running interface shown in the source report displays a temperature of 26 °C and a threshold of 64 °C; these are example values visible in the screenshot, not a general performance result.

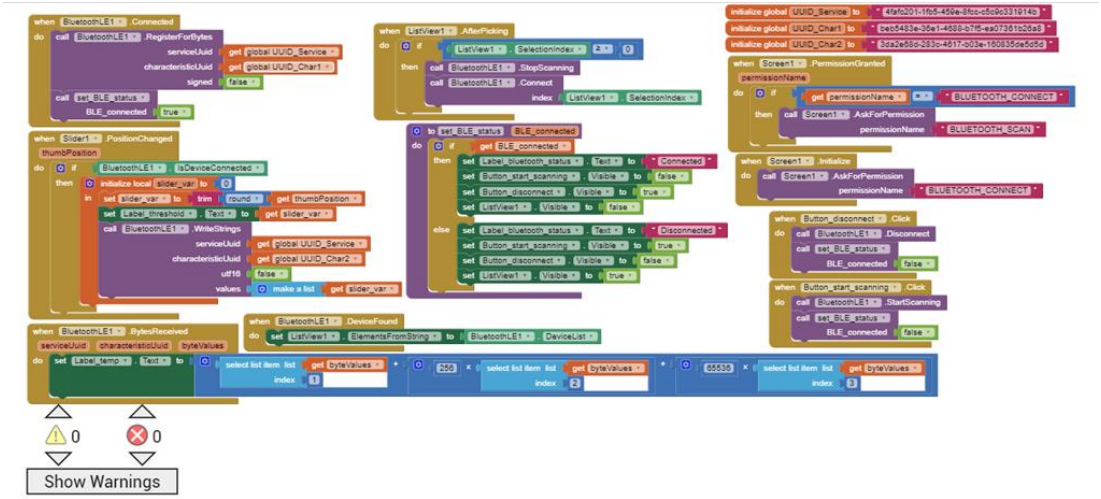


Figure 6. App Inventor logic blocks and example application screen

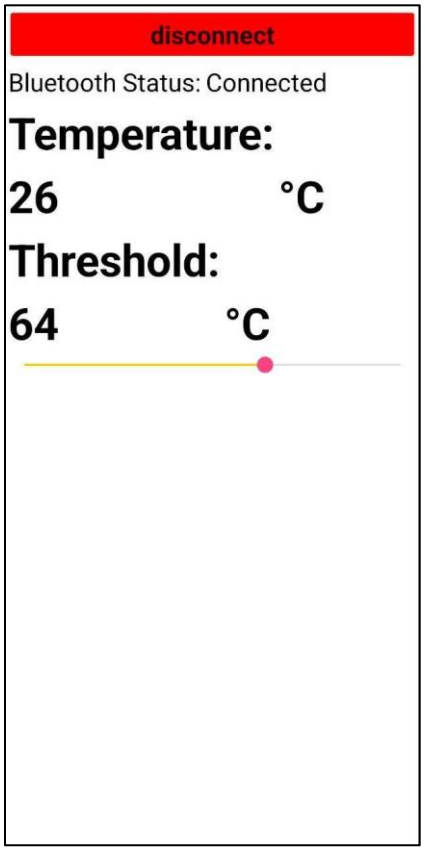


Figure 7. example application screen

6. Design Evidence and Results

6.1 Simulation Evidence

The supplied report presents a simulated output-voltage curve over the 0–100 °C temperature sweep. The plotted output decreases smoothly as temperature increases, consistent with the negative temperature coefficient of the NTC thermistor. The curve is used as evidence that the analog conditioning network produces a monotonic voltage response across the simulated interval.

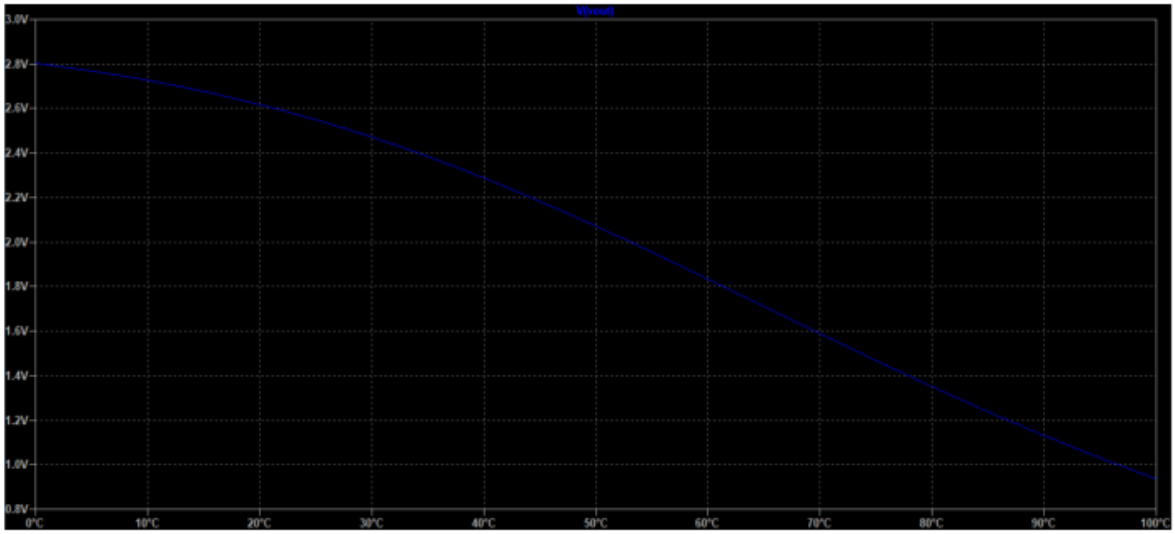


Figure 8. Simulated output-voltage response versus temperature, reproduced from the supplied project report.

6.2 Hardware and Software Evidence

- An Altium schematic is included in the original report.
- PCB routing and three-dimensional board views are included.
- The ESP32 firmware source code was supplied and reviewed for this report.
- Screenshots of the MIT App Inventor interface and its block logic are included.

6.3 Verification Status

The available evidence supports the completion of a circuit-design and simulation workflow, PCB design, firmware implementation, OLED interface, BLE GATT configuration, and mobile-app interface. However, the supplied materials do not include a tabulated set of physical temperature measurements, comparison against a calibrated reference thermometer, linearity or accuracy metrics, repeatability data, BLE range measurements, or power-consumption results. The report therefore distinguishes design and implementation evidence from experimentally validated metrological performance.

7. Limitations and Future Improvements

The following improvements would strengthen the project as an academic portfolio item and make the system easier to reproduce and evaluate:

- Calibration: record measurements at several known temperatures using a traceable reference thermometer, then report offset, gain, maximum error, and repeatability.
- Linearization assessment: provide the simulation or measurement data points and quantify the deviation from the fitted straight line across the stated operating range.
- ADC characterization: document the ADC attenuation and reference assumptions used by the ESP32, and verify the analog input voltage at the board.
- Signal stability: evaluate averaging or digital filtering and report the effect on response time and noise.
- Fault handling: define behavior for an open-circuit or short-circuit thermistor and for invalid BLE threshold values.
- BLE protocol: document the exact byte representation of the temperature notification and threshold write, including signedness, byte order, and units.
- Power and packaging: measure supply current and document the assembled PCB, connector pinout, and enclosure if applicable.
- Application reproducibility: include the MIT App Inventor project export or equivalent source artifact, subject to any library or extension licensing requirements.

8. Conclusion

This project integrates an NTC thermistor sensing network, analog signal conditioning, an ESP32 controller, an OLED display, BLE communication, and a mobile application into a compact temperature-monitoring system. The hardware design uses a parallel resistor to improve the thermistor's linearity, a voltage divider to scale the sensor signal, and a buffer stage to reduce loading. The ESP32 firmware reads the conditioned analog signal, converts it to temperature using the supplied linear transfer equation, displays the result locally, and communicates with a mobile application through BLE.

The project is a useful demonstration of multidisciplinary engineering work spanning sensor interfacing, analog electronics, PCB design, embedded programming, wireless communication, and user-interface development. To present it as a rigorously validated measurement instrument, the next step should be a documented calibration and test campaign, together with resolution of the component and firmware verification points listed in Appendix B.

Appendix A. Firmware Configuration and Key Code Excerpts

A.1 Required Libraries

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <BLEDevice.h>
#include <BLEServer.h>
#include <BLEUtils.h>
#include <BLE2902.h>
```

A.2 Pin and Display Configuration

```
Adafruit_SSD1306 IES(128, 64, &Wire, -1);

int led1 = 15;   // Red LED
int led2 = 4;    // Green LED
int opout = 13;  // Analog input
```

A.3 Temperature Conversion

```
float adc_count = analogRead(opout);
float temperature =
    adc_count * 3.3 / 4096.0 * (-53.561) + 149.97;
```

The variable name adc_count is used in this explanatory excerpt to make the raw ADC quantity explicit. The supplied source code uses the variable name voltage for this raw reading.

A.4 BLE Identifiers

```
#define SERVICE_UUID        "4fafc201-1fb5-459e-8fcc-c5c9c331914b"
#define CHARACTERISTIC_UUID "beb5483e-36e1-4688-b7f5-ea07361b26a8"
#define CHARACTERISTIC_UUID_2 "3da2e68d-283c-4617-b03e-160835de5d5d"
```

A.5 Main Firmware Responsibilities

Routine / callback	Responsibility
setup()	Initializes serial communication, GPIO, OLED, BLE server, service, characteristics, and advertising.
loop()	Reads the ADC, calculates temperature, updates LEDs and OLED, and sends notifications when connected.
MyServerCallbacks::onConnect()	Sets the connection-state flag when a BLE central connects.
MyServerCallbacks::onDisconnect()	Clears the connection-state flag after disconnection.
CharacteristicsCallBack::onWrite()	Converts the written threshold payload to an integer and stores it in pChar2_value_int.

Appendix B. Technical Verification Notes for the Project Team

The following points are not silently resolved in this report because the supplied materials contain differing or incomplete details. Confirming them against the assembled hardware and the final application will improve technical traceability.

Item	Evidence in supplied materials	Action before public release
Operational amplifier	The simulation figure labels LM324; the component-selection discussion names LM358.	Confirm the actual fitted IC and make the schematic, BOM, and report consistent.

Initial threshold	threshold is declared as 26, but the comparison uses pChar2_value_int, initialized to 0 and updated by BLE writes.	Decide whether a local default threshold is required; initialize the variable actually used in the comparison.
BLE payload format	The notification characteristic is assigned an integer value; the threshold callback converts incoming data from a string to an integer.	Document and test the exact data representation expected by the mobile application in both directions.
ADC conversion coefficients	The code uses −53.561 and 149.97 with the ADC count scaled by 3.3/4096.	Provide the calibration or curve-fitting data from which these coefficients were obtained.
Physical validation	The report contains simulation plots and design screenshots but no measurement table or accuracy analysis.	Add measured test points, reference-instrument comparison, error calculations, and repeatability results.
Temperature interval	The report discusses approximately 1–100 °C; the divider design uses the 0 °C endpoint for voltage-limit calculation.	State the final specified operating range and test the endpoints on the physical unit.